



ROP CHAINING

Smashing the Stack

About Me!



- WIMT Tech Intern
Did not know cyber program existed :(
- Georgia Tech BS/MS '28
Computing Systems,
SysArch + Cybersecurity
- Interests
Rhythm Games,
K-Pop, Larping, etc.

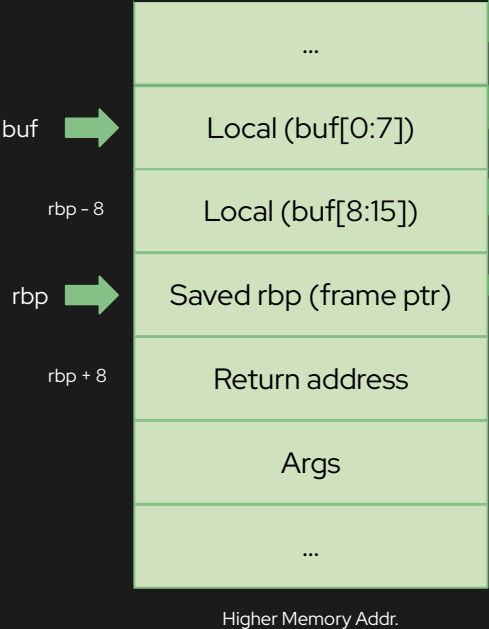


Presentation overview

- Intro to Buffer Overflow
- Demo + Image Setup
- Intro to ROP
- ROP Demo
- ROP Protections + Extension Activity
- Please ask questions!!

Intro to Buffer Overflows

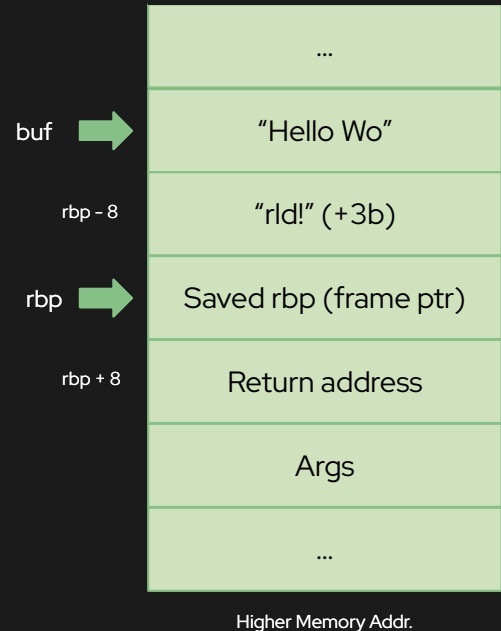
```
void vulnerable(char *filename)
{
    char buf[16];
    read_input(buf, filename);
}
```



Intro to Buffer Overflows

```
void vulnerable(char *filename)
{
    char buf[16];
    read_input(buf, filename);
}
```

Input: "Hello World!"

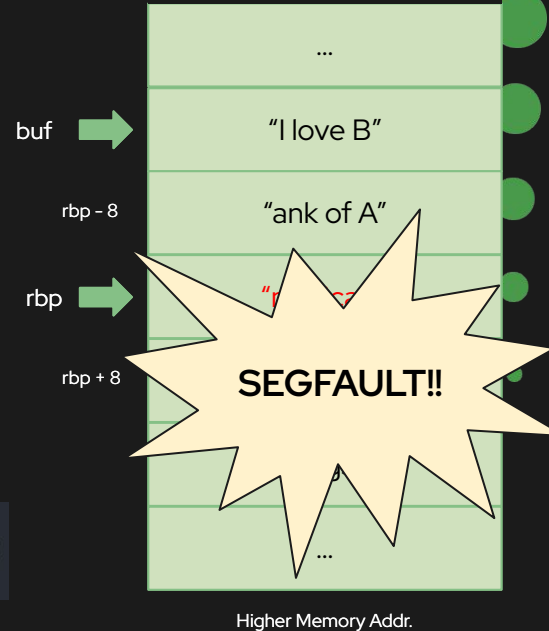


Intro to Buffer Overflows

```
void vulnerable(char *filename)
{
    char buf[16];
    read_input(buf, filename);
}
```

Input: "I love Bank of America so much!!"

```
Program received signal SIGSEGV, Segmentation fault.
0x000000000040191c in vulnerable (filename=0x7fffffff93d
3d "payload.bin") at ropvuln.c:13
```



GDB

Debugger for C programs! Lets you step through a program and view memory addresses, raw assembly, registers, etc.

```
(gdb) disas vulnerable
Dump of assembler code for function vulnerable:
0x00000000004018f2 <+0>:    endbr64
0x00000000004018f6 <+4>:    push    %rbp
0x00000000004018f7 <+5>:    mov     %rsp,%rbp
0x00000000004018fa <+8>:    sub     $0x20,%rsp
0x00000000004018fe <+12>:   mov     %rdi,-0x18(%rbp)
0x0000000000401902 <+16>:   mov     -0x18(%rbp),%rdx
0x0000000000401906 <+20>:   lea     -0x10(%rbp),%rax
0x000000000040190a <+24>:   mov     %rdx,%rsi
0x000000000040190d <+27>:   mov     %rax,%rdi
0x0000000000401910 <+30>:   mov     $0x0,%eax
0x0000000000401915 <+35>:   call    0x4018a7 <read_input>
0x000000000040191a <+40>:   nop
0x000000000040191b <+41>:   leave
=> 0x000000000040191c <+42>:   ret
```

disassemble

View the raw assembly for an input C function! Also shows addresses of each instruction.

breakpoint

Set a breakpoint at a certain memory address or instruction. Useful for testing!

x and print

View values associated with labels or memory addresses.

```
void win()
{
    printf("You won!\n");
}

void vulnerable(char *filename)
{
    char buf[16];
    read_input(buf, filename);
}
```



Buffer Overflow Demo

Set up the image with setup.sh and compile the code with build.sh

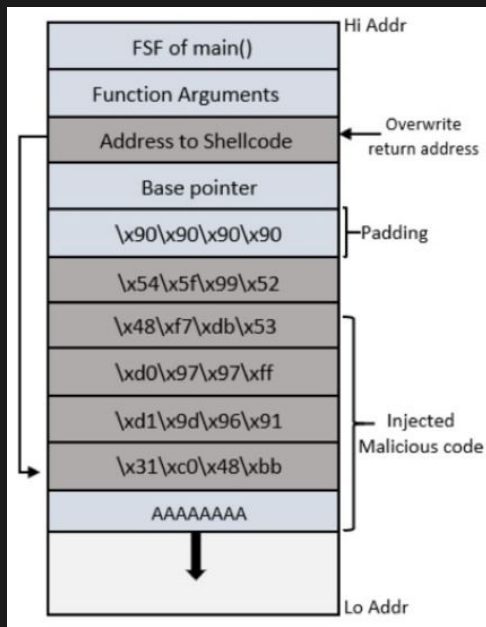
Let's try using GDB for a basic attack!

Attacks + Defenses: Cat and Mouse Game

Other Exploits

We can use a buffer overflow to:

- Redirect execution
- Override local variables
- **Execute arbitrary code!!**



Defenses

Modern Operating Systems implement **DEP** (Data Execution Prevention)

- Distinction between code + data; User data input can't be executed anymore
- Prevents direct shellcode injection
- What do we do now?

Solution: ROP Chaining!

Idea: If we can't execute code that we write, what if we just use code that's already been written?

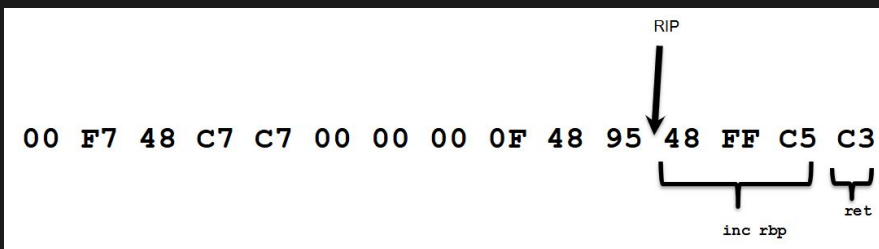
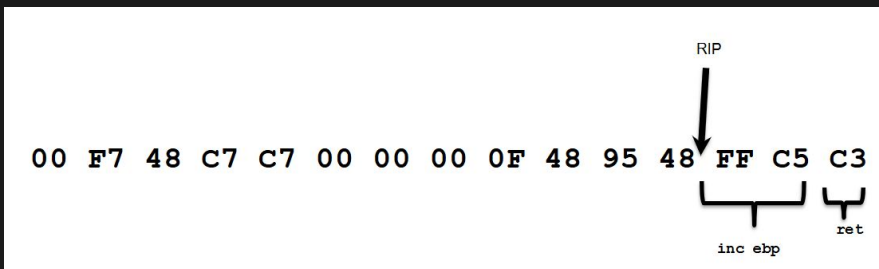
Problem: How can we get malicious code from benign binaries?



Fortunately, x86 gives us a way! :D

- x86 assembly has variable length instructions
- No fixed "offset" to start reading from
- The same block of bytes can be interpreted as different blocks of code!

In large codebases (i.e. with stdio) this means we can pretty much find **any single instruction we want!**



The Exploit: ROP Chaining

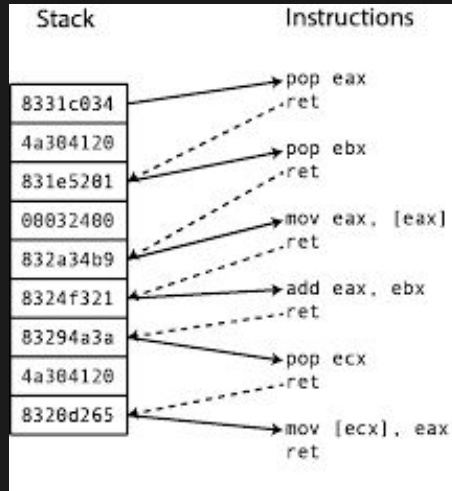
What is ROP?

Return-Oriented Programming

The Idea: We can “chain” together many small gadgets consisting of one desired instruction and a **return** instruction

Think of it like breaking a single program into a bunch of smaller programs!

```
func() -> func1() ret; func2() ret; func3() ret;
```



Applications

ROP essentially allows you to execute whatever code you want! Some common applications include:

Running a function with some privileged set of arguments

(Functions in x86 are pass-by-register, in the order rdi, rsi, rdx, etc.)

Executing arbitrary syscalls (very dangerous!) – privilege escalation, random execs, etc.

ROPgadget

Our tool for constructing ROP Chains! It automatically finds the addresses of binaries containing instructions we want.

```
root@d1d893bae61c:/ROP_Trainer# ROPgadget --binary
ropvuln --depth 3 | grep "pop rdi"
0x0000000000040a29a : pop rdi ; jmp 0x401060
0x0000000000040ed3e : pop rdi ; jmp 0x4010b0
0x00000000000416f0f : pop rdi ; jmp 0x4151a0
0x00000000000416f94 : pop rdi ; jmp 0x416110
0x0000000000041a943 : pop rdi ; jmp 0x41a270
0x000000000004082e5 : pop rdi ; jmp 0x41a5b0
0x000000000004773e6 : pop rdi ; jmp 0x41c770
0x0000000000041d103 : pop rdi ; jmp 0x41c940
```

Basic Usage

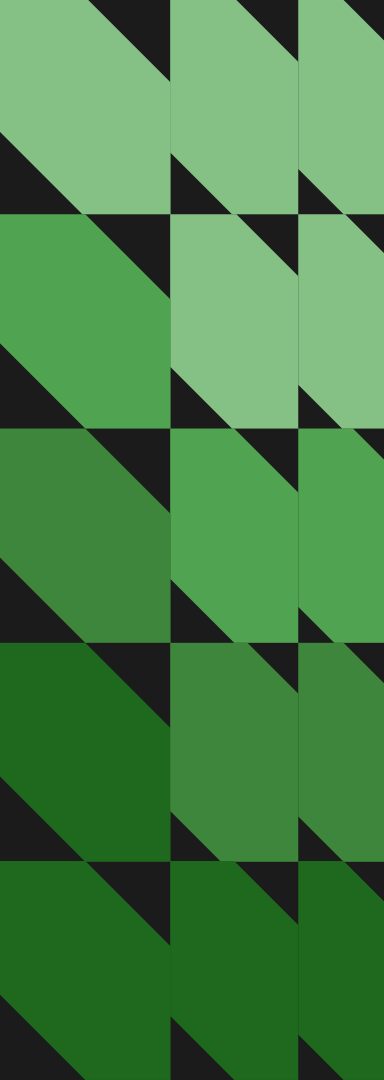
```
ROPgadget --binary [filename]
| grep "pop rdi"
```

depth

Depth flag limits the size of the gadgets returned (Sometimes needs to be larger to find one)

multibr

Returns gadgets with multiple branch instructions (i.e. syscall ; ret;)



```
const char admin_password[] = "wellsfargo";

void target(char *password, int password_len, int is_admin)
{
    if (is_admin != 1 || strncmp(password, admin_password, password_len) != 0)
    {
        printf("Authentication required, exiting now...\n");
        return;
    }

    printf("Welcome, admin!\n");
}
```

ROP Chaining Demo

Set up the image with setup.sh and compile the code with build.sh

Using ROPgadget to break into a simple admin login!

ROP IRL

ROP as we just studied isn't quite applicable in the real world now – in practice measures like ASLR and PIE are implemented to randomize the address space so we can't find fixed gadget addresses. Stack canaries also exist to prevent stack smashing altogether.

Of course, there are potential workarounds – see Heartbleed!

Thank you

Try out another exercise with privilege escalation on the demo repo!

https://github.com/jxchen21/ROP_Trainer

Recordings/Guides

[Challenge 0](#)

[Challenge 1](#)

[Challenge 2](#)